Introduction

This testbed can be used for cybersecurity research related to industrial control networks (ICS) with a focus on PLCs.

For further information:

- If you are not aware of the threats faced by ICS networks take a look at the motivation behind the development of this testbed and some reading recommendations.
- If you want to use the testbed refer to the usage chapter of this documentation.
- If you want to learn more about the technical details behind the testbed take a peek at them here.

Motivation

The goal of this testbed is to facilitate research related ICS security and educated IT personnel on the unique challenges of ICS security and OT personnel on any cyber threats that they might not be aware of. This is done by providing a demonstrator for attacks on PLC networks that contains components commonly found in ICS networks.

User Manual

This chapter serves as a user manual and guides through all steps required for the setup.

If you run into any issues not covered by this manual or see room for improvement, please reach out and we will improve.

Getting Started

The script `testbed.py` provides functionality such as setting dependencies (Docker, etc.), building the containers and starting/stopping.

Installation:

The installation script supports Debian, Ubuntu and ArchLinux. It is essentially a wrapper around the official instructions provided by Docker. The flag `--dont-ask-sudo` can be used to skip the script from asking everytime before running a sudo command.

```
./testbed.py --install debian # options are 'debian', 'ubuntu' and 'arch'
```

Building, Starting and Stopping the testbed

You can build the local version and start it the following way:

```
./testbed.py --build local
./testbed.py --start local -d
```

You can replace `local` with one of `{local, prod, dev}`, however most of the time you will want the local version. For more information look at the version chapter.

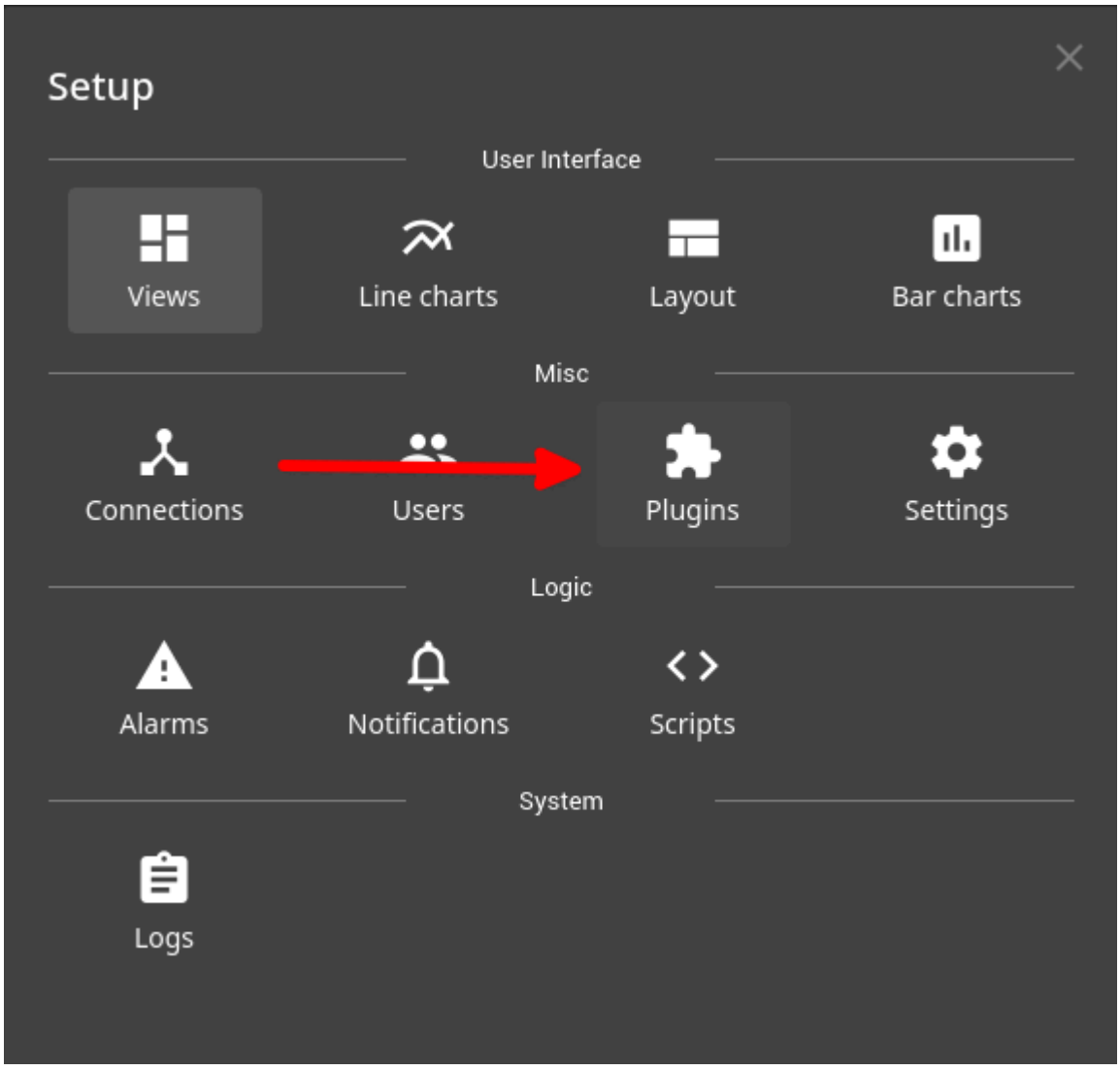
If you want to stop it again you can with:

```
./testbed.py --stop
```

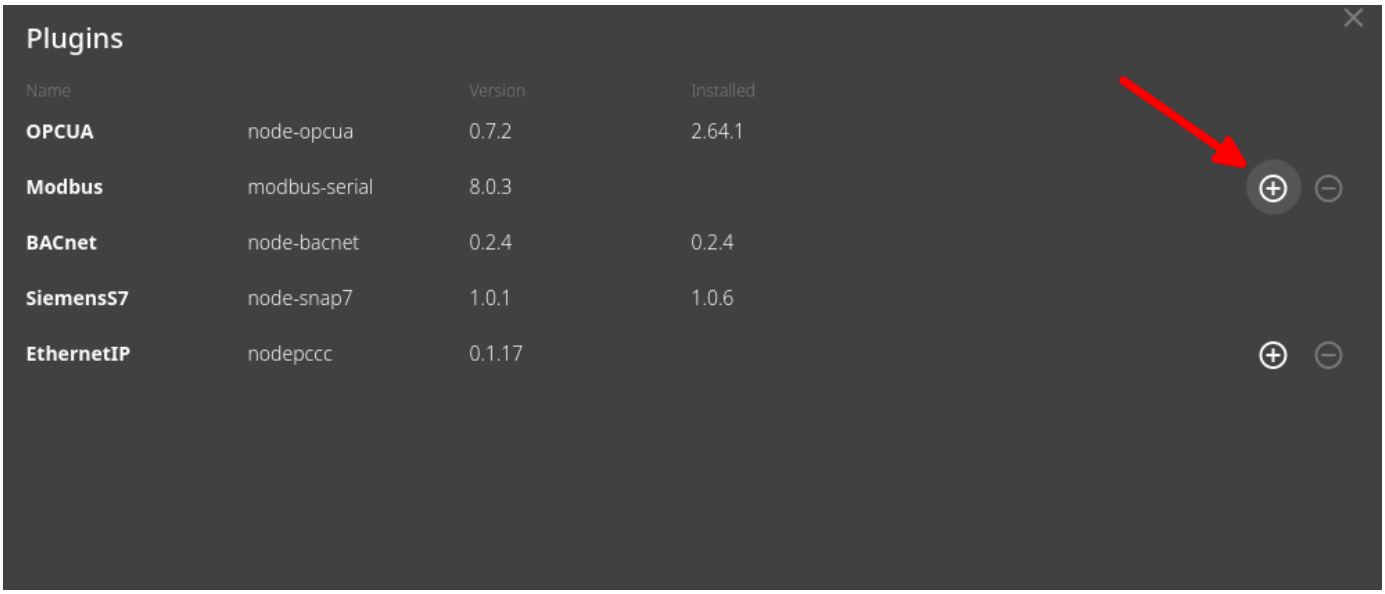
Setting up FUXA

1. Go to https://hmi-plctb.aisec.fraunhofer.de or http://localhost:4001
2. Press on the burger menu and click on editor.

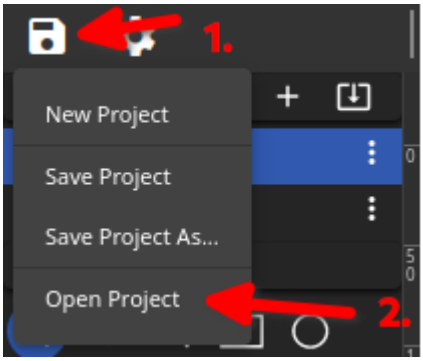
3. Press on the cogwheel in the upper left corner and select `Plugins`.



4. Enable the Modbus plugin and press OK.



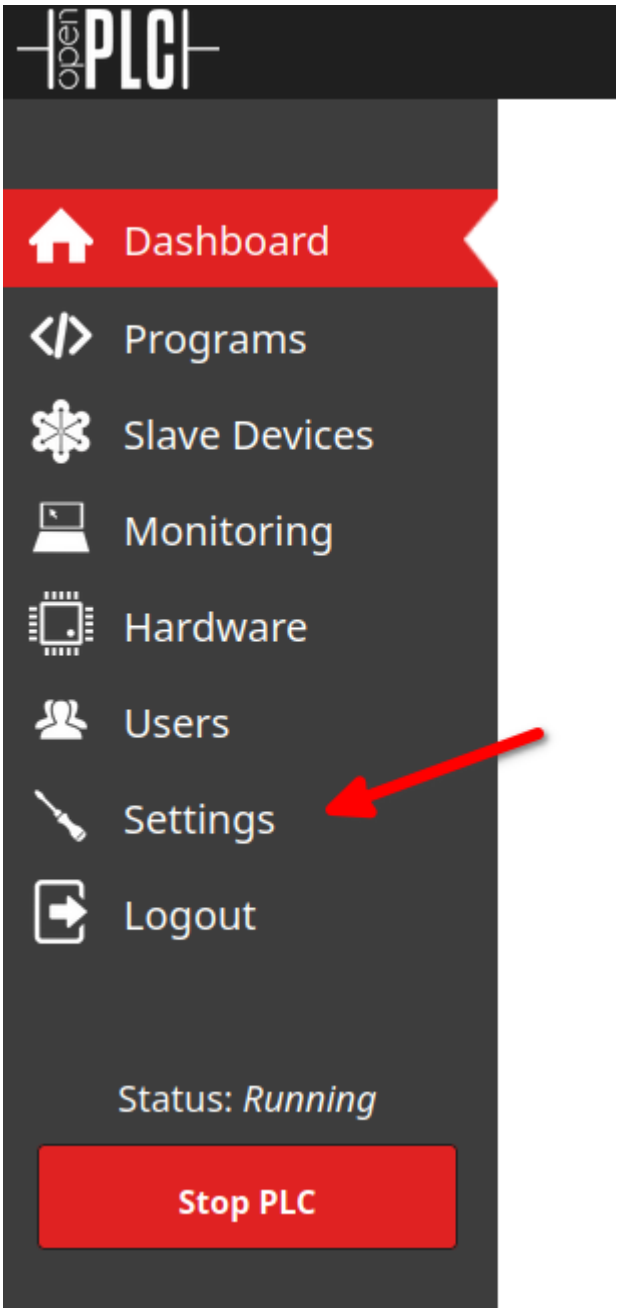
5. Press the floppy disk icon and select `Open Project` and navigate to the file `v3/FUXA/fuxa-config.json`.



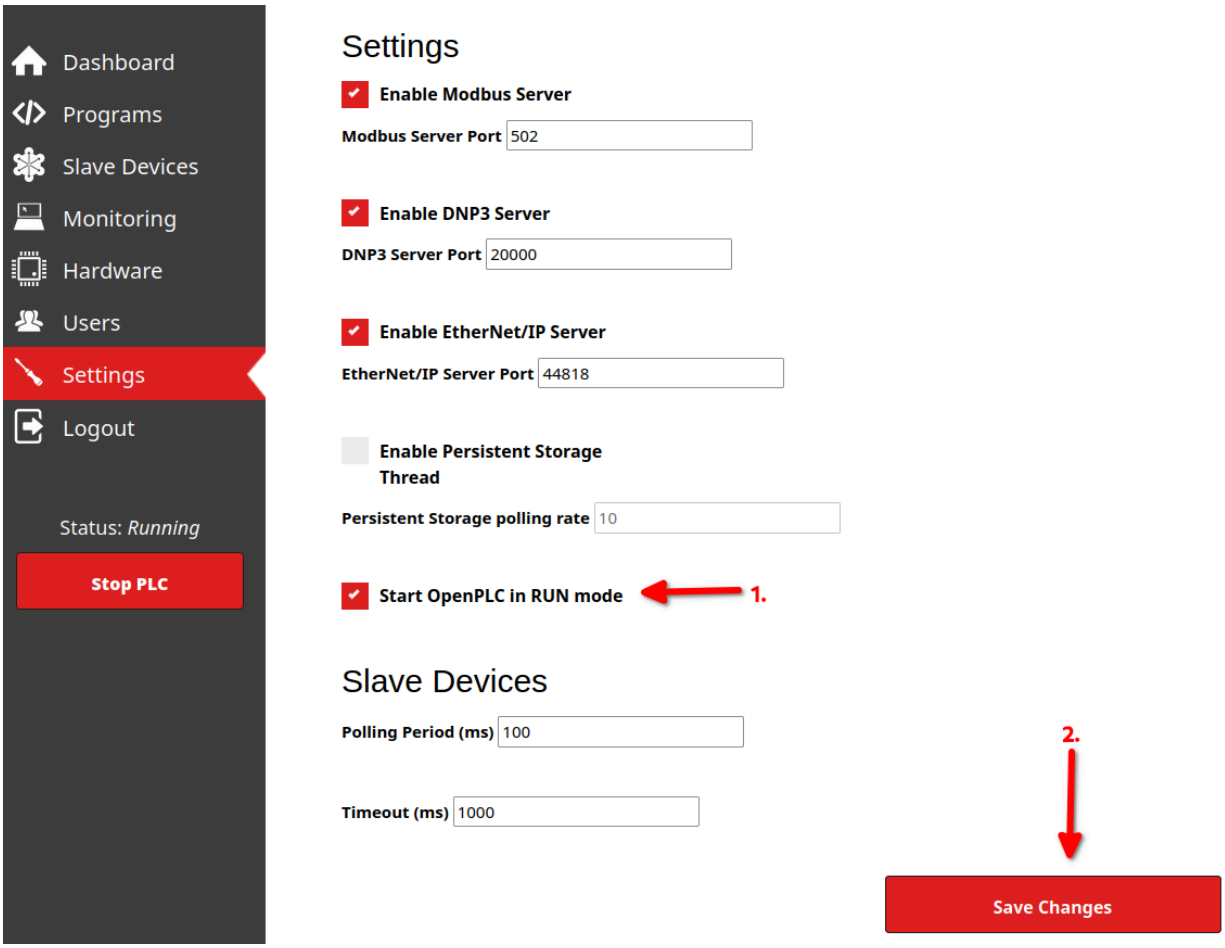
6. Press the burger menu and select `Lab`

Setting up OpenPLC

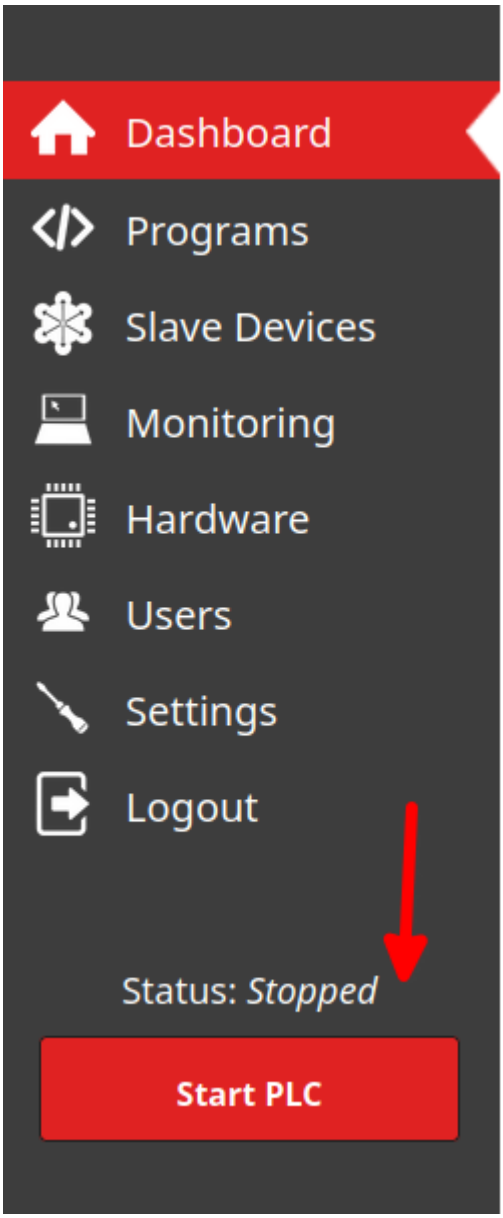
1. Go to https://plc-plctb.aisec.fraunhofer.de or http://localhost:4004
2. Login with default credentials, `openplc` for both username and password.
3. On the sidebar click on `Settings`.



4. Enable the toggle `Start OpenPLC in RUN mode`.



5. Press ’Start PLC’ button.



Additionally consult the OpenPLC documentation.

Setting up the Historian

1. In a browser open http://localhost:4002/ (local) or https://hist-plctb.aisec.fraunhofer.de (prod) **(Note: Chronograf sometimes has issues running correctly on Firefox, try using a Chromium based browser if you run into any issues.)**
2. Click through the Chronograf setup, no entries have to be changed. Skip setting up any dashboards at this moment.
    - for the database replace `localhost` with `influxdb`
    - for the Kapacitator replace `localhost` with `kapacitator`
3. On the left bar click on Dashboards.
4. Click 'Import Dashboard' and select the file from: `historian/chronograf/` for both Modbus and OPCUA.

Connecting to the Attacker

1. You can connect to the attacker at: https://attacker-plctb.aisec.fraunhofer.de/ or http://localhost:4005.

2. Follow the steps in the image:

Launching the Attacks

Modbus Modification Attack

```
# Assumes you are connected to the attacker.

cd /usr/src/exploits
python3 ./start_mitm_modification.py

# Terminate with Ctrl-C  when you want to stop the attack. (might require
multiple signals.)
```

What it does

- Begins with ARP cache poisoning.
- Modifies Modbus traffic between:
  - PLC <-> Industrial Process
  - PLC <-> Historian
- Leads to temperature values being underreported, which damages our simulated system.

Sniff OpenPLC Login Credentials.

1. Execute the following:

```
# Assumes you are connected to the attacker.

cd /usr/src/exploits
python3 ./password_sniff.py
```

2. Wait for someone to login to the OpenPLC webinterface with the credentials.
3. The terminal of the attacker will print out the credentials, which were sniffed from unencrypted HTTP traffic.

What it does

- ARP cache poisoning, allowing us to inspect HTTP traffic.
- The attacker has obtained the credentials for further attacks.
- Alternative: use dictionary attack against weak credentials which are very common.

Upload Malicious Program and Blind Historian

Assumptions

- The attacker has obtained the credentials to the PLC (look above on how to obtain them).
- The attacker has created a malicious PLC program which is available here.. If you want to restore the default program you can use this one.

How to launch it.

1. Go to the OpenPLC web interface.
2. Go to the programs section.
3. Upload the malicious program. (`heater-malicious.st`)
4. Wait for it to compile (currently slightly slow due to increased compile time due to open62541 usage) and start the PLC again.
5. In an attacker terminal run the following:

```
# Assumes you are connected to the attacker.

cd /usr/src/exploits
python3 ./start_mitm_modification_hist_plc.py

# Terminate with Ctrl-C when you want to stop the attack.
```

6. Observe how values are reflected depending on protocol.

What it does

- PLC behvaior is altered and it ignores too high temperature values.
- Begins with ARP cache poisoning to hide malcious behavior of PLC.
- Modifies Modbus traffic between: PLC <-> Historian
- Modbus based historian loses vision of real values.
- OPC UA based HMI and historian maintain true and umodified vision (compare here).
- Leads to unsafe temperature values being underreported, which damages our simulated system.

Credentials

OpenPLC

For now, we use the OpenPLC default credentials:

- username: `openplc`
- password: `openplc`

Attacker

- For shellinabox:
  - username: `attacker`
  - password: `attacker`
- For the KaliLinux container:
  - username: `root`
  - password: `root`

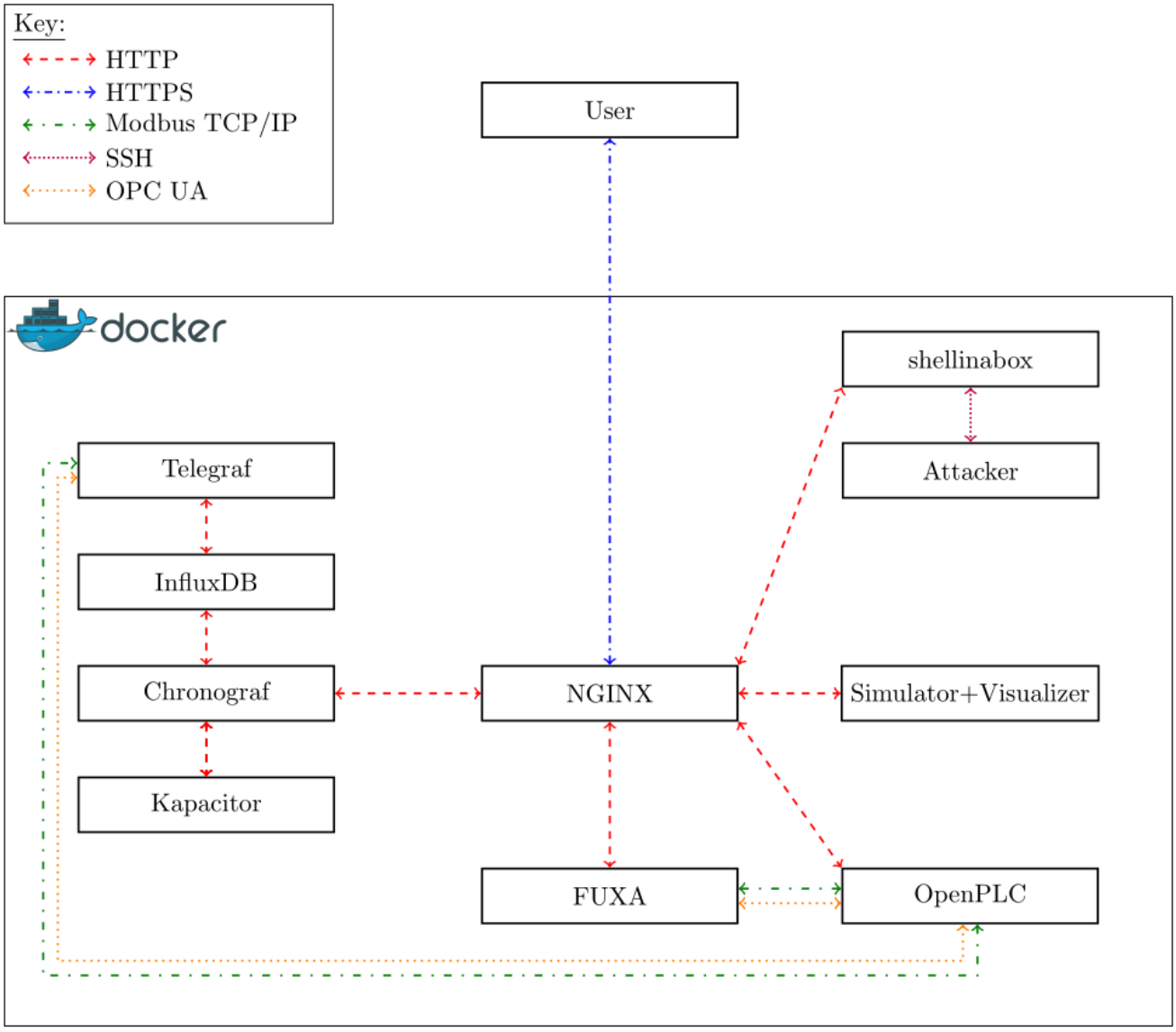
Different Versions

Technical Details

This chapter covers the technical details of the testbed. Beginning with what components are used, followed by the used industrial protocols. Afterwards, it details how docker is used, and finally it describes how authentication and the PKI is managed.

Summaryc

This image briefly sums up what components exist in the testbed and what kind of protocols are used.



The Components

The testbed uses of open-source components and libraries to address important aspects of control networks.

OpenPLC

OpenPLC is a fully open-source PLC, that can either be run on microcontrollers or as a software-PLC. It targets the IEC 61131-3 standard for PLCs and supports the following programming languages: Ladder Logic (LD), Function Block Diagram (FBD), Instruction List (IL), Structured Text (ST), and Sequential Function Chart (SFC). These programs can be written with OpenPLC Editor.

OpenPLC was forked for this testbed, in order to provide OPC UA support. The reason for this fork was the lack of an open-source PLC that supports OPC UA. OPC UA functionality uses the open62541 library. Complying with the PLCOpen companion standard(s) was not the goal of this project and is left for future work.

The project website can be found here. The source-code of the fork that is used in this testbed can be found here.

HMI (FUXA)

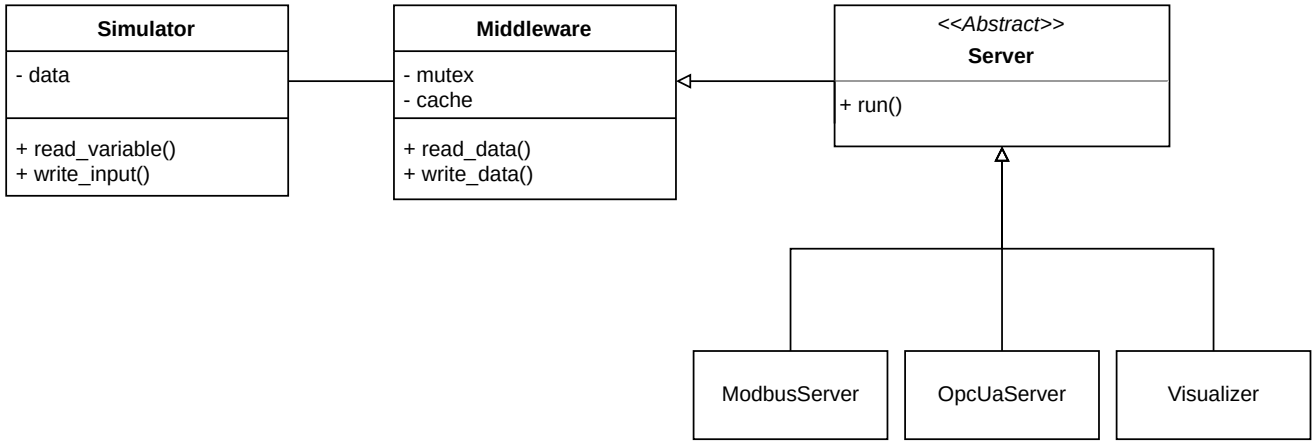
The HMI software is FUXA. Which has a Node.js backend and an Angular frontend. It can connect to: Modbus RTU/TCP, Siemens S7 Protocol, OPC-UA, BACnet IP, MQTT, Ethernet/IP (Allen Bradley). More information can be found in the GitHub repository.

Historian (TICK Stack)

The historian is built with the TICK stack (Telegraf, InfluxDB, Chronograf, Kapacitor). InfluxDB is a time-series database, Telegraf is a data-collection agent and Chronograf is a dashboard software. Telegraf uses Modbus and OPC UA to gather data from the PLC and stores it in the database, which is then visualized with Chronograf.

Physical Process Simulator and Visualizer

Physical processes are an essential part of a control network, realizing them with real-world equipment is unfeasible in many cases, due to being very expensive. The physical process of our testbed is entirely virtual and implemented with software simulation. The simulator is written Python and very modular, it is shown in the following image.



The simulation itself is handled by the `Simulator` object, the `Middleware` implements an interface to access the data from the simulation and handles synchronization and caching (in order to reduce load on the simulation model). The abstract `Server` can be used to extend the simulator with new protocols, we implemented this for Modbus (using the PyModbus library) and OPC UA (using the asyncua library).

Additionally, we also implemented a simple web application that visualizes the physical process independently of any outside interference (such as attackers).

All of the components run in their own thread(s).

Attacker

The attacker host is a Kali Linux container to which we can connect via SSH.

Libraries

The testbed used a number of open source libraries, including open62541, PyModbus and asyncua.

open62541

`open62541` is one of the major open-source implementations of OPC UA, it is platform independent and written in the common subset of C99 and C++98. It is certified by the OPC Foundation. The library is a low-level library, which allows high customizability and low-level control.

As mentioned, it is used in the OpenPLC fork maintained for this testbed.

The project is housed here, the documentation can be found here and the source code here.

PyModbus

The simulator uses the PyModbus library to implement a Modbus server. The documentation can be found here and the GitHub repository here.

asyncua

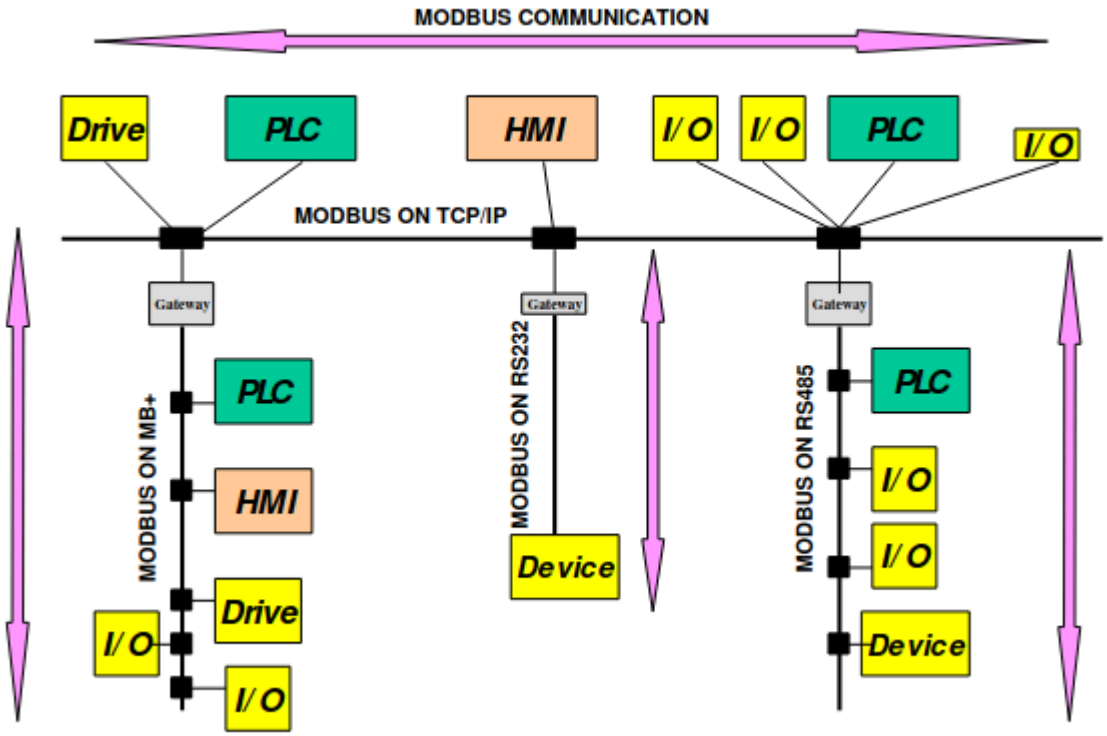
`asyncua` is used implement the OPC UA server of the simulator. The github repository can be found here.

Industrial Protocols

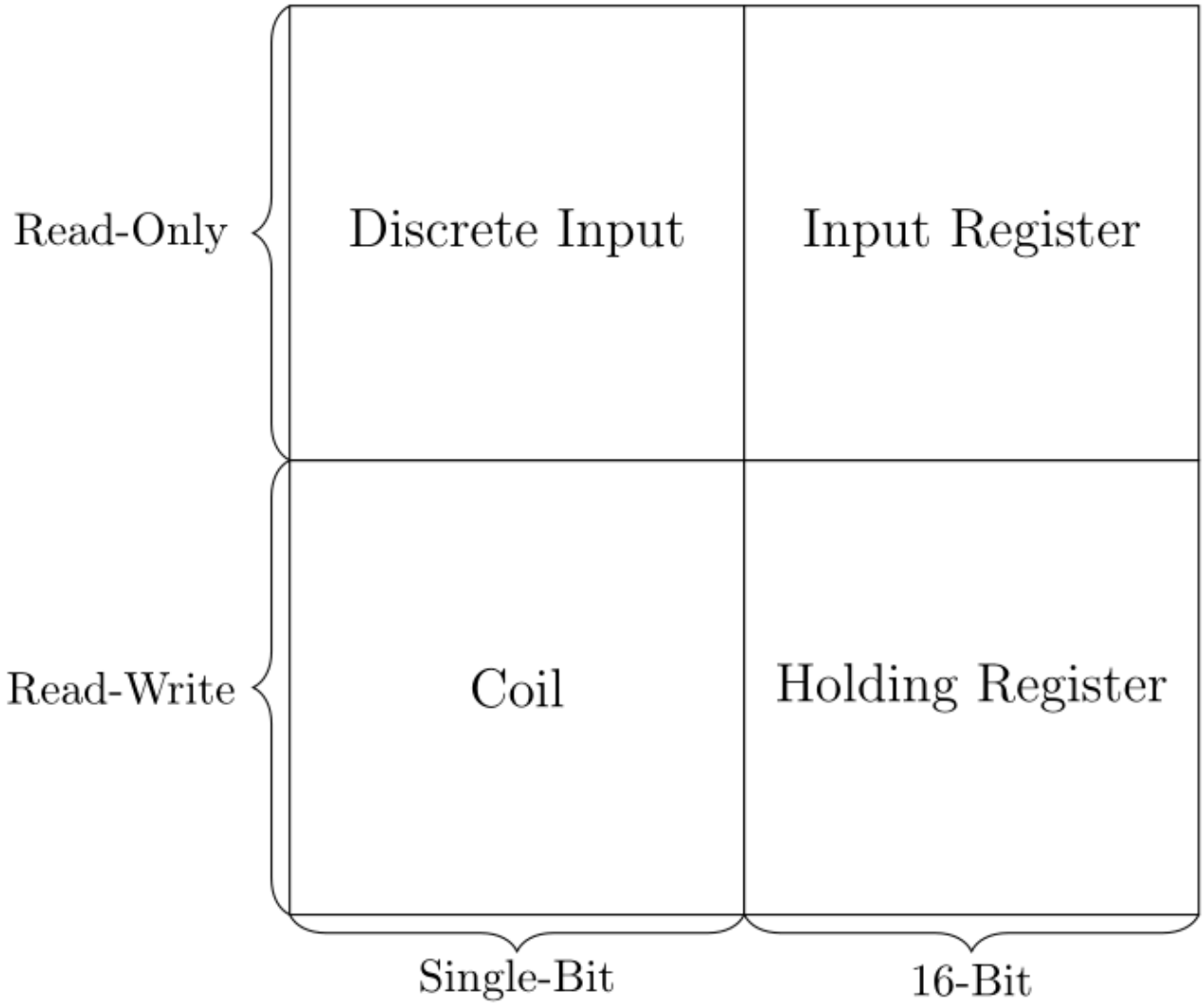
The testbed uses two major industrial protocols, Modbus and OPC UA. The former is a quite old protocol, and has been used in industry for many decades. The latter is newer and has been gaining popularity in the industrial world in the context of the 4th industrial revolution (Industry 4.0).

Modbus

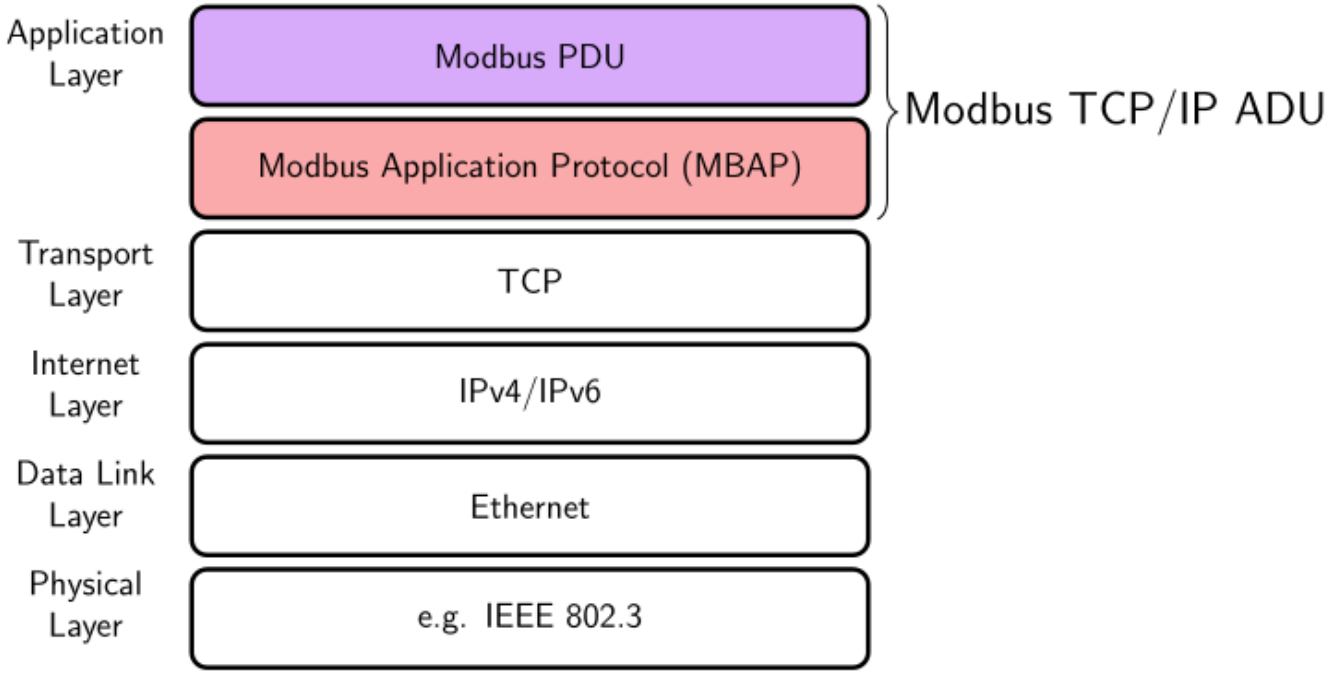
Modbus is a server-client protocol that can be used at the field level. The following image shows possible use cases, it is taken from the Modbus Specification.



It structures data into four different tables: coils, discrete inputs, input registers and holding registers. Each have different data sizes and access rights (for clients). This is illustrated by the following picture.



Modbus can be used on top of different protocols stacks, such as serial connections or TCP/IP. The serial version is called Modbus RTU and the TCP/IP version is simply called Modbus TCP/IP. The protocol slightly differs depending on the protocol stack.



Issues with Modbus

Due to a number of reasons, Modbus suffers from multiple problems.

The first issue is a complete lack of cryptography in the base protocol, which misses encryption and message authentication. The second is the very rudimentary level of access control, which (in the base protocol) is limited to the read/write access of the different tables. A mechanism that is more suitable to data modelling rather than proper access control. Furthermore, it also allows a very low degree of data modelling due to being restricted into using the four previously mentioned tables. This also affects scalability as Modbus servers with many data points can become unwieldy to use.

This is caused by it being a very old protocol, developed during a time where control networks faced different constraints. There are initiatives to address issues related to security, for instance, the 'Modbus Security Protocol'. It adds cryptography and role-based access control. However, this improvement requires retro-fitting and has not been widely adopted.

Resources

- All specifications related to Modbus are available here.
- How Modbus is used in OpenPLC can be found here (server) and here (client)

OPC Unified Architecture OPC UA (IEC62541)

Background

OPC UA is a powerful protocol that aims to standardize communication for industrial applications, for both vertical but also machine-to-machine communication.

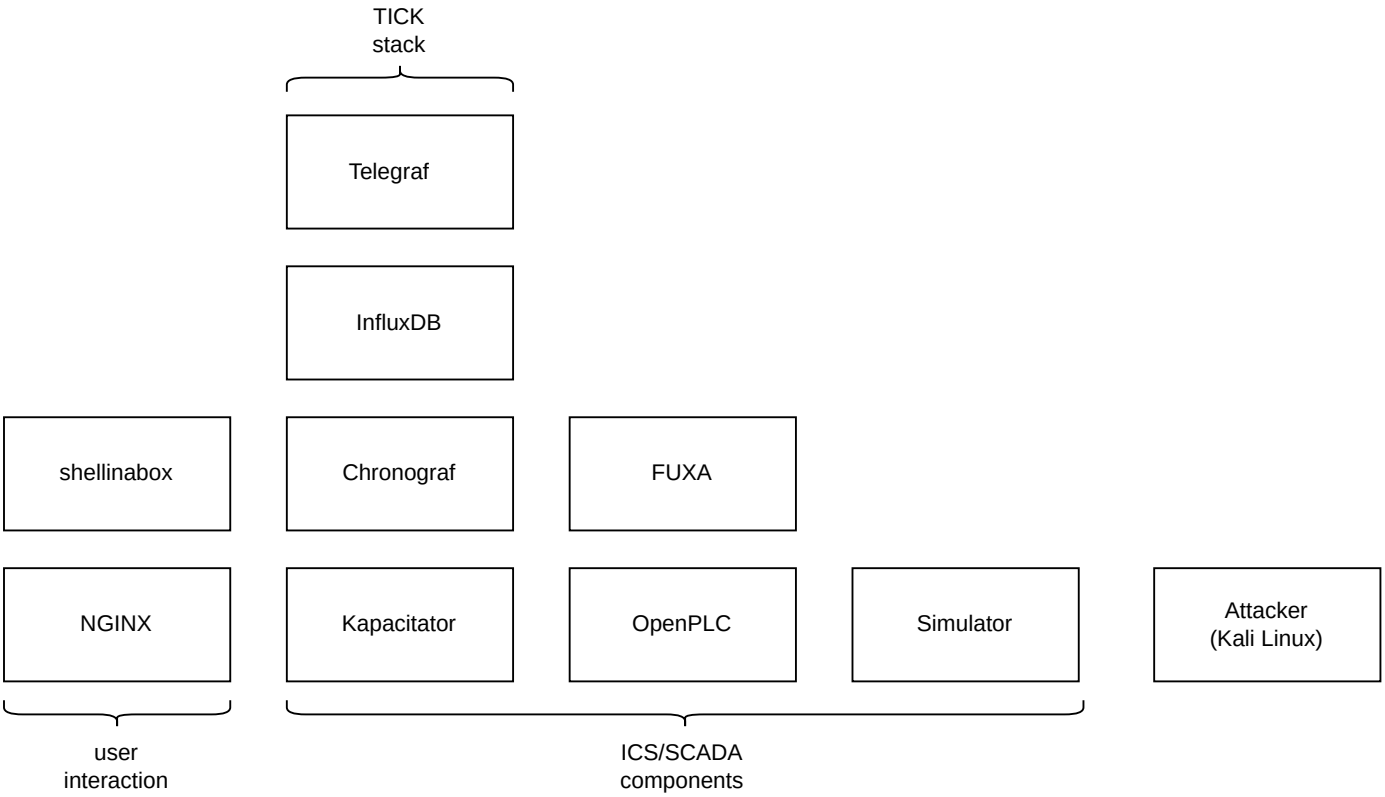
Security

The protocol offers many security mechanisms, such as message authentication and encryption. But also powerful fine-grained access control.

Virtualization Details

The testbed is virtualized with Docker and docker-compose. Using Docker gives us great flexibility and usability without sacrificing too much fidelity.

Containers



NGINX Container

The NGINX container is used to mediate access to individual testbed components, it works as a reverse proxy. It allows us to authenticate access and protect remote traffic to the testbed with HTTPS. Which is not supported by multiple testbed components by themselves.

Linux Capabilities

Some containers require elevated Linux capabilities (reference) in order to operate.

Container	Capability	Reason
OpenPLC	SYS_NICE	increase process priority of processes related PLC runtime
OpenPLC	IPC_LOCK	lock memory related to PLC runtime
Attacker	NET_ADMIN	enable promiscuous mode
Attacker	NET_RAW	open raw sockets, required for ScaPy

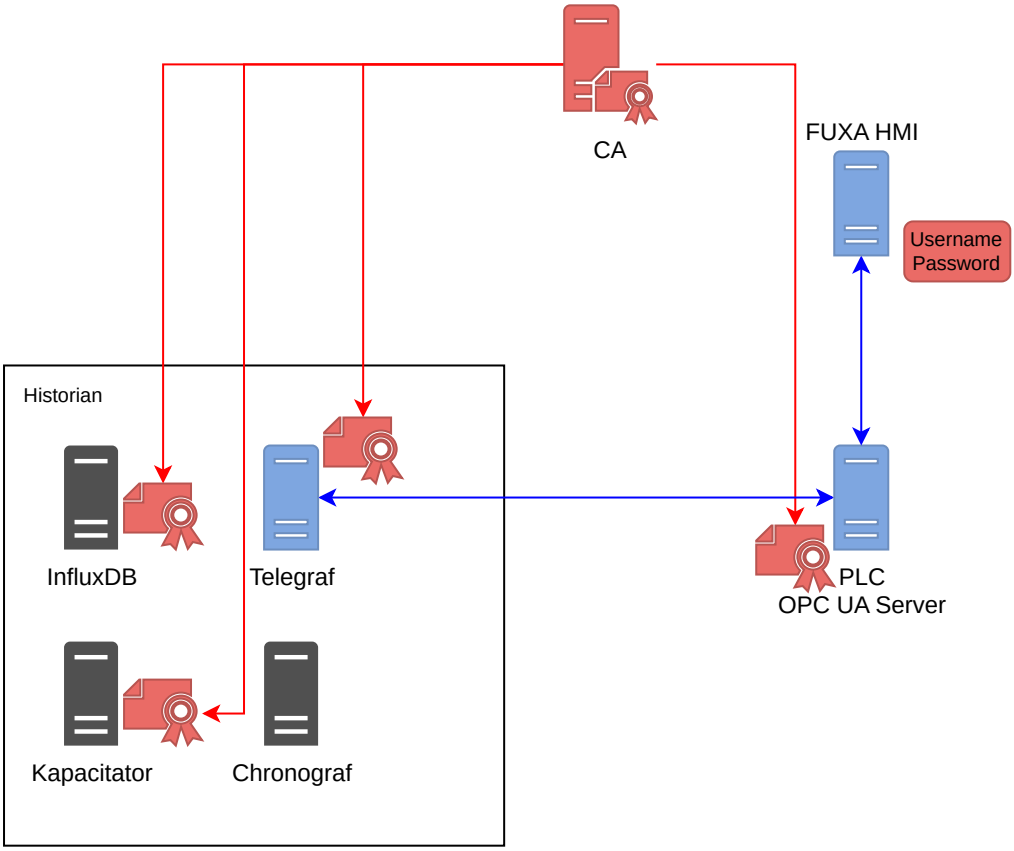
Certificates, Authentication and Acess Control

The testbed uses a PKI for purposes of authentication and message protection. FUXA (HMI) does not use certificates for OPC UA, due to a lack of support.

PKI

The PKI was implemented with **OpenSSL**. It has a root certificate authority (CA) and an intermediate CA, that signs the application certificates. The root CA is only used to sign the intermediate CA and potentially revoke it.

It issues the following certificates:

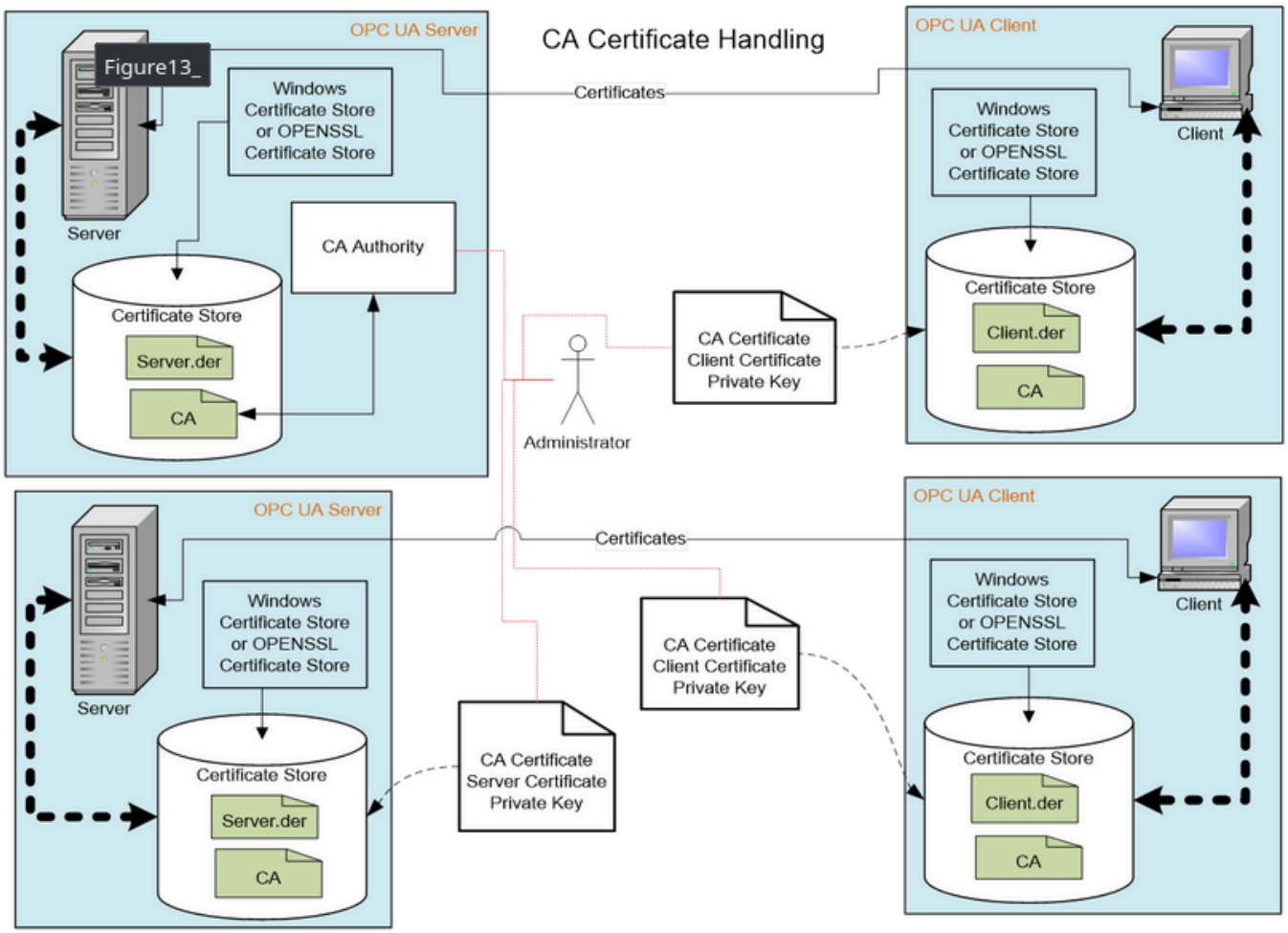


Authentication

The certificates are used to authenticate the OPC UA clients and servers. As well as TICK stack components mutually. As mentioned, all these certificates are signed by the intermediate CA.

For OPC UA **all CAs have to maintain a CRL**, these are distributed to the applications alongside the CA certificate.

The following image (taken from the OPC UA specification) again shows are certificates are supposed to be used in OCP UA.



Access Control

We use the certificates to identify the clients that connect to the server. The identities are then internally mapped to roles. Each of these roles has different capabilities. We define three different roles:

- observers: read only access
- operators: can change values for input variables, e.g. turn the controller logic on and off.
- admins: unrestricted rights.

In our case we give the HMI the operator role and the historian an observer role.

Additionally, we also restrict writing to variables is undesirable. For instance, writing to sensor values aggregated by the PLC obviously is something that we don't want to allow.

References

For more details on how OPC UA uses certificates refer to:

- https://reference.opcfoundation.org/v104/Core/docs/Part2/8/
- Chapter 2.3 of 'Praxishandbuch OPC UA' (2nd edition, German, ISBN 978-3-8343-3454-1)

OPC UA based Security Mechanism

Within the testbed we use the mechanism OPC UA already provides in order to secure the network of our small industrial control system. As mentioned in this previous section OPC UA already provides many mechanism on its own. Which is a big step forward from previous generations of industrial protocols, which lacked many of these and either had to retrofit them or left them out entirely. The security of OPC UA has been analyzed by the German Federal Office for Information Security (BSI), with results confirming good design regarding the security of the protocol.

This section describes the mechanisms themselves in more detail and how we use OPC UA's mechanisms.

Access Control

OPC UA provides mechanisms for access control to each nodes. This allows control over which users are allowed to start a session and which nodes they can access.

User Authentication

OPC UA has four different kind of authentication mechanism:

- **no authentication:** also called anonymous login, actions cannot be mapped to actors and users are not authenticated in any way whatsoever.
- **X.509 Certificates:** these kind of certificates are used widely, in many technologies and can be used as part of public-key-infrastructure in order to establish trust top-down from trusted root-certificates or by trusting individual self-signed certificates. Users have a certificate and the corresponding secret private key, and are authenticated based on proving their knowledge of it.
- **username/password:** another widely used method of authentication, that needs no introduction.
- **JSON Web Token (JWT)**: a method of authentication that is very popular in the world of web development. Listed here only for the sake of completeness, as it is not supported by our used OPC UA library (open62541) and is not widely supported by other libraries either.

These mechanism are used to create sessions.

User Authentication inside the Testbed

We use certificates and username/password based authentication in the testbed for critical nodes. Where possible we use certificate-based authentication and username/password otherwise.

All application certificates are signed by a CA, which serves as the trust anchor and is manually distributed to the hosts along with a certificate revocation list. This CA is outlined here. With OPC UA, it would also be possible to use self-signed certificates, by adding certain rejected certificates to the trust store, however this approach has drawbacks such as complicated trust revocation issues and scales badly with a big number of applications and corresponding certificates. OPC UA also defines another way of certificate management via the use of a GlobalDiscoveryServer (GDS), which are however out of the scope of this testbed.

Access for non-authenticated users is restricted. Allowing session

Per-Node Access Control

OPC UA also allows us to limit which nodes can be access by which user. For instance, we can hide confidential information from third parties which might only need limited access to an OPC UA server, by hiding these nodes from them and preventing access. This could be a vendor providing remote administration or maintenance services.

We can also control read/write access, for instance, it would make little sense to allow the modification of sensor values. Whereas it would make a lot of sense to allow certain users the modification of input values, such as goal temperatures and others.

Per-Node Access Control is still work in progress in the testbed.

Message Protection

The protection of messages is affected by two properties of a session, the *SecurityMode* and the *SecurityPolicy*.

There are three different security modes:

- None: no cryptographic protection
- Sign: messages are signed, this ensures their authenticity and integrity. However, this does not protect against eavesdropping.
- SignAndEncrypt: messages are a signed and encrypted, providing additional protection against eavesdropping.

There are many different security policies, these among others affect the utilized cryptographic algorithms used for signing and encryption, can mandate the usage of certificate. Some standardized security policies are not recommended for use anymore. One example would be Basic256, due to using SHA1, which is not considered secure anymore. Additionally, there is also the 'None' mode which provides no security at all and is only compatible with the 'None' security mode.

In the testbed we use SignAndEncrypt in combination with the most secure security polices supported by our OPC UA library (open62541).

Further References

- https://reference.opcfoundation.org/v104/Core/docs/Part2/5.2.3/
- https://reference.opcfoundation.org/v104/Core/docs/Part2/4.9/
- Practical Security Guidelines for Building OPC UA Applications (1)
- Practical Security Recommendations for building OPC UA Applications (2)
- Exploring OPC UA Security Concepts
- BSI - OPC UA Security Analysis

Literature and other Resources

Developer Manual

This part of the documentation explains how to develop with the testbed.

Folder Structure

The repository has the following structure.

```
├── attacker:                          Files related to the attacker
├── certificates:                      Everything related to the CA and
certificate management
├── docker-compose.dev.yml:            Makes the components accessible outside
the docker network.
├── docker-compose.external.yml        Redirects some connections to outside the
docker networks.
├── docker-compose.prod.yml:           Adds the webserver to the base setup.
├── docker-compose.yml:                Base compose file.
├── docs:                              Sources related to this documentation.
├── FUXA:                              Files related to the HMI
├── historian:                         Files related to the TICK stack
├── industrial-process:                Files related to the process simulator and
visualizer
├── nginx:                             Files related to the webserver
├── PLC:                               Files related to OpenPLC
├── readme.md:                         This file.
```

Simulator

Technical Details:

The simulation itself is handled by the `Simulator` object, the `Middleware` implements an interface to access the data from the simulation and handles synchronization and caching (in order to reduce load on the simulation model). The abstract `Server` can be used to extend the simulator with new protocols, we implemented this for Modbus (using the PyModbus library) and OPC UA (using the asyncua library).

Additionally, we also implemented a simple web application that visualizes the physical process independently of any outside interference (such as attackers).

All of the components run in their own thread(s).

Extension

If you want to modify the simulation code you can do this in: `industrial-process/simulator`.

Historian

The historian uses the TICK (Telegraf, InfluxDB, Chronograf and Kapacitator) stack. If one wants to improve the testbed the most interesting component is Telegraf, because it is the only one that directly interacts with the others.

Telegraf Plugins

We use the following plugins. Consult these links to change the setup.

- Modbus plugin
- OPCUA plugin
- InfluxDB v1 plugin

HMI (FUXA)

Updating the HMI interface.

If you want to update the HMI you need to use the FUXA Editor.

Updating the data collection.

You can add/edit connections, to use other protocols.

PLC

Changing the PLC Program.

Use the OpenPLC editor to update the PLC program.

Working with the OpenPLC fork.

To work on the fork checkout the repository which homes it.